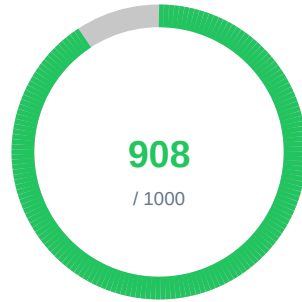


Code Hardener

ATTESTATION CERTIFICATE



bulletproof-compliance-portal

Scan cdc64f | Branch: main | Jul 24, 2026

Attestation ID

1f7054ea-96ad-420f-87d5-...

Type

in-toto

Subject

bulletproof-compliance-portal

Created

Jul 24, 2026, 6:37 PM

Subject Digest

sha256:31048f31bb2043ff526f124c7e7cadd099ee5136314c1ea46a9f40a2...

Signature Status

CRYPTOGRAPHICALLY SIGNED

Algorithm: Ed25519 (Local Signing Key)

Signature

JpqKAp6vhBHGIjBJ6coowbDdErOb1G13QStrchWYlUsG10PSuIqrLa7Aj0u3hv6f
C8C4BQQSJwX8s302ZmcBAA==

This attestation certifies the security scan was performed and results were recorded with cryptographic integrity.

Scan Report — bulletproof-compliance-portal

Branch: main | Status: completed | Jul 24, 2026

908 / 1000 Score — Quality: excellent

Scan Details

Profile: standard
Duration: 31s
Started: Jul 24, 2026, 6:37 PM
Completed: Jul 24, 2026, 6:37 PM
Files Analyzed: 266
Scanners Run: 10 of 12
Open Findings: 31
Resolved / Suppressed:0
Total Detected: 31
Codebase: Python: 116 files | Markdown: 36 files | Config: 4 files | JavaScript/TypeScript: 3 files | Docker: 2 files | Shell: 1 files

0 Critical 0 High 19 Medium 11 Low 1 Info

Scanner Results

12 tools executed — 5 passed | 5 with open findings | 1 not applicable | 1 errors

Scanner	Category	Outcome	Findings	Duration	Files Scanned
trivy	SCA	ERROR	0	2s	266
gitleaks	Secrets	PASS	0	0s	266
opengrep	SAST	FAIL	2	11s	13
checkov	IaC	PASS	0	7s	6
grype	SCA	FAIL	7	7s	4
syft	SBOM	FAIL	10	1s	10

package-validator	AI Code Quality	PASS	0	0s	1
oxlint	Code Quality	N/A	0	0s	3
ruff	Code Quality	FAIL	11	0s	116
actionlint	CI/CD Security	PASS	0	0s	4
jscpd	Code Quality	PASS	0	0s	119
typos	Code Quality	FAIL	1	0s	266

Scanner Errors

trivy: Unexpected end of JSON input

Scanner Evidence & Audit Trail

gitleaks (Secrets)

PASS

Scan Scope: Working directory scanned for 150+ secret patterns with entropy analysis

Configuration: Custom gitleaks.toml

Files Analyzed: 266

Rules Checked: 150

Checks Performed:

- AWS credential detection
- API key detection
- Private key detection
- Generic secret patterns
- High-entropy string analysis
- Token detection

Standards: CWE-798, CWE-259, OWASP Top 10 (A07)

Methodology:

Regex-based pattern matching against 150+ secret patterns with entropy analysis. Scans current file contents and optionally git commit history. Applies allowlists and custom rules to reduce false positives. Secrets are redacted in output.

opengrep (SAST)

FINDINGS

Scan Scope: Multi-language SAST with auto, security-audit, owasp-top-ten, cwe-top-25, r2c-security-audit, and secrets rulesets

Configuration: Custom semgrep-rules.yml + default rulesets

Files Analyzed: 13

Rules Checked: 1

Checks Performed:

- OWASP Top 10 patterns
- Security audit rules
- Secrets detection
- Injection patterns
- XSS patterns
- Insecure crypto usage

Standards: OWASP Top 10, CWE Top 25, SANS Top 25

Methodology:

Pattern-based static analysis using Semgrep rule engine with auto, security-audit, owasp-top-ten, and secrets rulesets. Matches AST patterns against known vulnerability signatures.

checkov (IaC)

PASS

Scan Scope: IaC security analysis — 0 files, 0 checks (0 passed, 0 failed), frameworks: auto-detected

Configuration: Custom .checkov.yaml

Files Analyzed: 6

Checks Performed:

- Terraform misconfigurations
- CloudFormation template issues
- Kubernetes manifest hardening
- Dockerfile best practices
- Helm chart security
- GitHub Actions security
- GitLab CI security
- Secrets in IaC files

Standards: CIS Benchmarks, NIST SP 800-53, SOC 2, PCI-DSS, HIPAA

Methodology:

Graph-based static analysis of infrastructure-as-code files. Parses IaC definitions, builds resource graphs, and checks 1000+ built-in policies covering security, networking, encryption, and compliance.

grype (SCA)

FINDINGS

Scan Scope: SCA analysis: 7 unique CVEs across 4 vulnerable packages (filesystem)

Configuration: Grype with --by-cve deduplication and --add-cpes-if-none

Files Analyzed: 4

Rules Checked: 7

Checks Performed:

- CVE vulnerability matching
- Dependency version analysis
- CPE-based matching
- Advisory cross-referencing
- Fix availability detection

Standards: CVE/NVD, OWASP Top 10 (A06), GHSA

Methodology:

Generates SBOM using Syft, then matches all identified packages against vulnerability databases (NVD, GitHub Advisories, OS vendor databases). Reports CVEs with severity, fix versions, and references.

syft (SBOM)

FINDINGS

Scan Scope: SBOM of 10 packages across 1 language(s): python

Configuration: Syft with JSON + CycloneDX output for SBOM standards compliance

Files Analyzed: 10

Rules Checked: 4

Checks Performed:

- SBOM generation (Syft JSON + CycloneDX)
- License compliance analysis
- Copyleft license detection (AGPL, GPL, MPL)
- Weak copyleft classification (LGPL)
- Unknown license flagging
- Package URL (PURL) generation
- CPE generation

Standards: NTIA SBOM Minimum Elements, CycloneDX, SPDX, Executive Order 14028

Methodology:

Catalogs software packages by parsing package manifests, lock files, and installed package databases. Generates standardized SBOM in CycloneDX or SPDX format for supply chain visibility.

package-validator (AI Code Quality)**PASS****Scan Scope:** Validated 11 packages across 1 dependency file(s)**Configuration:** Concurrency: 10, Timeout: 5000ms per request**Files Analyzed:** 1**Rules Checked:** 1**Checks Performed:**

- npm registry existence validation
- PyPI registry existence validation
- Go module proxy existence validation
- Slopsquatting risk detection
- Hallucinated dependency identification

Standards: CWE-829, OWASP A08:2021, SLSA Supply Chain Security**Methodology:**

Parses dependency manifests and validates each package exists in its respective registry (npm, PyPI, Go proxy) via HTTP HEAD requests. Non-existent packages are flagged as hallucinated — a common AI code generation artifact that creates supply chain attack vectors.

oxlint (Code Quality)**N/A****Scan Scope:** JavaScript (.js, .jsx) and TypeScript (.ts, .tsx) source files**Files Analyzed:** 3**Checks Performed:**

- Code correctness checks
- Suspicious code detection
- Performance anti-patterns
- Import/export validation

Standards: CWE, ESLint Core Rules**Methodology:**

Rust-based linter that performs AST analysis of JS/TS files at high speed. Detects correctness issues, suspicious patterns, and performance anti-patterns.

ruff (Code Quality)**FINDINGS****Scan Scope:** Python linting and security analysis via Ruff**Configuration:** Default Ruff rules**Files Analyzed:** 116**Checks Performed:**

- Pyflakes error detection
- pycodestyle enforcement
- flake8-bandit security checks
- isort import ordering
- pep8-naming conventions
- pyupgrade modernization
- flake8-bugbear bug detection
- flake8-comprehensions optimization

Standards: PEP 8, PEP 257, CWE, OWASP Top 10**Methodology:**

Rust-based Python linter implementing 800+ rules from flake8, pylint, isort, and others. 10-100x faster than traditional Python linters.

actionlint (CI/CD Security)**PASS****Scan Scope:** .github/workflows/*.yaml files**Files Analyzed:** 4

Checks Performed:

- Workflow syntax validation
- Action version pinning checks
- Shell script validation
- Expression type checking
- Permissions analysis

Standards: GitHub Actions Security Best Practices, CWE-78

Methodology:

Static analysis of GitHub Actions workflow files. Validates syntax, checks for unpinned action versions, analyzes shell scripts for errors, and verifies expression types.

jscpd (Code Quality)

PASS

Scan Scope: All source files across supported languages

Files Analyzed: 119

Checks Performed:

- Cross-file duplicate detection
- Code clone identification
- Duplication percentage calculation

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Token-based analysis to detect copy-pasted code blocks across files and languages. Reports duplication percentage and identifies specific cloned regions.

typos (Code Quality)

FINDINGS

Scan Scope: All files in project directory

Files Analyzed: 266

Checks Performed:

- Source code spelling analysis
- Identifier typo detection
- Comment typo detection
- String literal typo detection

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Fast, low-false-positive spell checker designed for source code. Checks identifiers, strings, and comments against a curated dictionary.

Open Findings (31)

ruff

Code Quality | 11 findings | Ultra-fast Python linter (pyflakes, pycodestyle, flake8-bandit)

Severity	Title	File	CWE / Rule
MEDIUM	Ruff F401: `pathlib.Path` imported but unused `pathlib.Path` imported but unused	tests/test_signing.py:18	RUFF-F401
Fix:	Auto-fix available: Remove unused import: `pathlib.Path` (applicability: safe)		
MEDIUM	Ruff F401: `os` imported but unused `os` imported but unused	tests/test_signing.py:17	RUFF-F401
Fix:	Auto-fix available: Remove unused import: `os` (applicability: safe)		
MEDIUM	Ruff F401: `json` imported but unused	tests/test_signing.py:16	RUFF-F401

`json` imported but unused

Fix: Auto-fix available: Remove unused import: `json` (applicability: safe)

MEDIUM Ruff F401: `app.config.Config` imported but unused tests/test_annual_scheduler.py:422 RUFF-F401
`app.config.Config` imported but unused
Fix: Auto-fix available: Remove unused import: `app.config.Config` (applicability: safe)

MEDIUM Ruff F401: `os` imported but unused tests/test_annual_scheduler.py:12 RUFF-F401
`os` imported but unused
Fix: Auto-fix available: Remove unused import: `os` (applicability: safe)

MEDIUM Ruff F401: `urllib.parse.quote` imported but unused app/retention.py:101 RUFF-F401
`urllib.parse.quote` imported but unused
Fix: Auto-fix available: Remove unused import: `urllib.parse.quote` (applicability: safe)

MEDIUM Ruff F401: `datetime.timedelta` imported but unused app/reports.py:14 RUFF-F401
`datetime.timedelta` imported but unused
Fix: Auto-fix available: Remove unused import: `datetime.timedelta` (applicability: safe)

MEDIUM Ruff F401: `fastapi.responses.JSONResponse` imported ... app/main.py:25 RUFF-F401
`fastapi.responses.JSONResponse` imported but unused
Fix: Auto-fix available: Remove unused import: `fastapi.responses.JSONResponse` (applicability: safe)

MEDIUM Ruff F401: `hashlib` imported but unused app/main.py:16 RUFF-F401
`hashlib` imported but unused
Fix: Auto-fix available: Remove unused import: `hashlib` (applicability: safe)

MEDIUM Ruff F401: `httpx` imported but unused app/dsr_cascade.py:19 RUFF-F401
`httpx` imported but unused
Fix: Auto-fix available: Remove unused import: `httpx` (applicability: safe)

MEDIUM Ruff F401: `gzip` imported but unused app/dsr_cascade.py:9 RUFF-F401
`gzip` imported but unused
Fix: Auto-fix available: Remove unused import: `gzip` (applicability: safe)

grype

SCA | 7 findings | Dependency vulnerability scanner for containers and filesystems

Severity	Title	File	CWE / Rule
MEDIUM	CVE-2026-49452: Vulnerability in weasyprint@68.1 WeasyPrint has CSS Injection via Presentational Hints	/requirements.txt	CVE-2026-49452
Package: weasyprint Version: 68.1 Type: python Language: python Fix: Review this finding and apply the appropriate fix based on the description: WeasyPrint has CSS Injection via Presentational Hints OWASP: A06:2021-Vulnerable and Outdated Components			
MEDIUM	GHSA-gj48-438w-jh9v: Vulnerability in bleach@6.1.0 Bleach clean() / Cleaner() fails to sanitize dangerous URI schemes in allowed formaction attributes	/requirements.txt	GHSA-gj48-438w-jh9v

Package: bleach
Version: 6.1.0
Type: python
Language: python

Fix: Update bleach to version 6.4.0

OWASP: A06:2021-Vulnerable and Outdated Components

MEDIUM CVE-2025-71176: Vulnerability in pytest@8.3.3
pytest has vulnerable tmpdir handling

/requirements.txt

CVE-2025-71176

Package: pytest
Version: 8.3.3
Type: python
Language: python

Fix: Update pytest to version 9.0.3

OWASP: A06:2021-Vulnerable and Outdated Components

MEDIUM CVE-2024-56201: Vulnerability in jinja2@3.1.4
Jinja has a sandbox breakout through malicious filenames

/requirements.txt

CVE-2024-56201

Package: jinja2
Version: 3.1.4
Type: python
Language: python

Fix: Update jinja2 to version 3.1.5

OWASP: A06:2021-Vulnerable and Outdated Components

MEDIUM CVE-2025-27516: Vulnerability in jinja2@3.1.4
Jinja2 vulnerable to sandbox breakout through attr filter selecting format method

/requirements.txt

CVE-2025-27516

Package: jinja2
Version: 3.1.4
Type: python
Language: python

Fix: Update jinja2 to version 3.1.6

OWASP: A06:2021-Vulnerable and Outdated Components

MEDIUM CVE-2024-56326: Vulnerability in jinja2@3.1.4
Jinja has a sandbox breakout through indirect reference to format method

/requirements.txt

CVE-2024-56326

Package: jinja2
Version: 3.1.4
Type: python
Language: python

Fix: Update jinja2 to version 3.1.5

OWASP: A06:2021-Vulnerable and Outdated Components

LOW GHSA-8rfp-98v4-mm6: Vulnerability in bleach@6.1.0
Bleach: URI sanitization allows disallowed URI schemes with Unicode > U+00A0 in output

/requirements.txt

GHSA-8rfp-98v4-mm6

Package: bleach
Version: 6.1.0
Type: python
Language: python

Fix: Update bleach to version 6.4.0
OWASP: A06:2021-Vulnerable and Outdated Components

opengrep

SAST | 2 findings | Multi-language static analysis for security vulnerabilities

Severity	Title	File	CWE / Rule
MEDIUM	github-actions-mutable-action-tag	...rget/github/workflows/ci.yml:12	CWE-1357: Reliance on
GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-github-action compromises. Pin the reference to a full 40-character commit SHA instead, e.g. `uses: actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608`.			

```
requires login
```

Fix: Review this finding and apply the appropriate fix based on the description: GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-gi

OWASP: A08:2021 - Software and Data Integrity Failures | Rule: yaml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag

MEDIUM	github-actions-mutable-action-tag	...rget/github/workflows/ci.yml:11	CWE-1357: Reliance on
GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-github-action compromises. Pin the reference to a full 40-character commit SHA instead, e.g. `uses: actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608`.			

```
requires login
```

Fix: Review this finding and apply the appropriate fix based on the description: GitHub Actions step uses a mutable tag or branch reference. Tags and branch names can be silently repointed by the action owner, enabling supply-chain attacks — as seen in the trivy-action and kics-gi

OWASP: A08:2021 - Software and Data Integrity Failures | Rule: yaml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag

syft

SBOM | 10 findings | Software bill of materials generator for supply chain visibility

Severity	Title	File	CWE / Rule
LOW	Unknown License: uvicorn@0.34.0	/requirements.txt	SBOM-LICENSE-UNKNOWN
Package uvicorn has no license information. This may indicate a compliance risk.			

```
Package: uvicorn
Version: 0.34.0
Type: python
```

Fix: Verify the license for this package before using in production.

OWASP: A08:2021-Software and Data Integrity Failures

LOW	Unknown License: qdrant-client@1.12.1	/requirements.txt	SBOM-LICENSE-UNKNOWN
Package qdrant-client has no license information. This may indicate a compliance risk.			

```
Package: qdrant-client
Version: 1.12.1
Type: python
```

Fix: Verify the license for this package before using in production.

OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: pyyaml@6.0.2 /requirements.txt SBOM-LICENSE-UNKNOWN
Package pyyaml has no license information. This may indicate a compliance risk.

Package: pyyaml
Version: 6.0.2
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: pydantic@2.10.4 /requirements.txt SBOM-LICENSE-UNKNOWN
Package pydantic has no license information. This may indicate a compliance risk.

Package: pydantic
Version: 2.10.4
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: jinja2@3.1.6 /requirements.txt SBOM-LICENSE-UNKNOWN
Package jinja2 has no license information. This may indicate a compliance risk.

Package: jinja2
Version: 3.1.6
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: httpx@0.28.1 /requirements.txt SBOM-LICENSE-UNKNOWN
Package httpx has no license information. This may indicate a compliance risk.

Package: httpx
Version: 0.28.1
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: fastapi@0.115.6 /requirements.txt SBOM-LICENSE-UNKNOWN
Package fastapi has no license information. This may indicate a compliance risk.

Package: fastapi
Version: 0.115.6
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: cryptography@48.0.1 /requirements.txt SBOM-LICENSE-UNKNOWN
Package cryptography has no license information. This may indicate a compliance risk.

Package: cryptography
Version: 48.0.1
Type: python

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: actions/setup-python@v5 /github/workflows/ci.yml SBOM-LICENSE-UNKNOWN
Package actions/setup-python has no license information. This may indicate a compliance risk.

Package: actions/setup-python
Version: v5
Type: github-action

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: actions/checkout@v4 /github/workflows/ci.yml SBOM-LICENSE-UNKNOWN
Package actions/checkout has no license information. This may indicate a compliance risk.

Package: actions/checkout
Version: v4
Type: github-action

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

typos

Code Quality | 1 finding | Source code spell checker

Severity	Title	File	CWE / Rule
INFO	Typo: "unparseable" should be "unparsable" Found typo "unparseable" in typo. Suggested correction: unparsable.	app/main.py:136	TYPOS-SPELLING
Fix:	Replace "unparseable" with "unparsable".		

Codebase Coverage (266 files analyzed)

This scan analyzed 266 files using 10 security tools across 8 testing categories. Each scanner targets files relevant to its analysis domain (e.g., SAST scanners analyze source code, SCA scanners analyze dependency manifests).

Files by Language:

Python: 116 files	Markdown: 36 files
Config: 4 files	JavaScript/TypeScript: 3 files
Docker: 2 files	Shell: 1 files

Files by Extension:

.py: 116	.html: 94	.md: 26
.txt: 10	.other: 3	.js: 3
.dockerfile: 2	.conf: 2	.css: 2
.example: 1	.yaml: 1	.dockerignore: 1
.toml: 1	.gitignore: 1	.sh: 1
.yaml: 1	.json: 1	

Per-Scanner File Coverage:

Scanner	Category	Files Analyzed	Scope
gitleaks	Secrets	266	Working directory scanned for 150+ secret p...

opengrep	SAST	13	Multi-language SAST with auto, security-aud...
checkov	IaC	6	IaC security analysis — 0 files, 0 checks (...)
grype	SCA	4	SCA analysis: 7 unique CVEs across 4 vulner...
syft	SBOM	10	SBOM of 10 packages across 1 language(s): p...
package-validator	AI Code Quality	1	Validated 11 packages across 1 dependency f...
oxlint	Code Quality	3	JavaScript (.js, .jsx) and TypeScript (.ts,...)
ruff	Code Quality	116	Python linting and security analysis via Ruff
actionlint	CI/CD Security	4	.github/workflows/*.yaml files
jscpd	Code Quality	119	All source files across supported languages
typos	Code Quality	266	All files in project directory

Files with Findings (9):

File Path	Findings	Scanners
/requirements.txt	15	grype, syft
tests/test_signing.py	3	ruff
app/main.py	3	ruff, typos
app/dsr_cascade.py	2	ruff
/.github/workflows/ci.yml	2	syft
tests/test_annual_scheduler.py	2	ruff
/scan-target/.github/workflows/ci.yml	2	opengrep
app/reports.py	1	ruff
app/retention.py	1	ruff

Software Bill of Materials (10 packages)

Package types: python: 8 | github-action: 2

Top licenses: Unknown (10)

Package	Version	Type	Language	License
actions/checkout	v4	github-actio	-	Unknown
actions/setup-python	v5	github-actio	-	Unknown
cryptography	48.0.1	python	python	Unknown
fastapi	0.115.6	python	python	Unknown
httpx	0.28.1	python	python	Unknown
jinja2	3.1.6	python	python	Unknown
pydantic	2.10.4	python	python	Unknown
pyyaml	6.0.2	python	python	Unknown
qdrant-client	1.12.1	python	python	Unknown
uvicorn	0.34.0	python	python	Unknown